

SOFTWARE TESTING & VALIDATION · FINAL PROJECT

EasyService: Test Summary Report & Foundations Reflection — Parts H & I (v1.0)

Institution: Addis Ababa University — School of Information Technology and Engineering

Course: Software Testing and Validation

Document: Test Summary Report & Foundations Reflection (Parts H & I)

Repository: <https://github.com/Abyman2/Easyservice>

Group Members

#	Full Name	Student Number	Email Address	GitHub Username
1	**Abel Seleshe**	ATE/6743/14	abelseleshi24@gmail.com	[Abyman2](https://github.com/Abyman2)
2	**Baheran Tesfaye**	ATE/5750/14	Bahrantesfaye1@gmail.com	[Bahrann](https://github.com/Bahrann)
3	**Nebiyu Yohannes**	ATE/3973/14	natijhonny@gmail.com	[NebiyuYohannes](https://github.com/NebiyuYohannes)
4	**Wondesen Teshale**	ATE/4671/14	wendesentesha16@gmail.com	[WondesenTeshale](https://github.com/WondesenTeshale)

1. Executive Summary & Release Recommendation

This Test Summary Report evaluates the software quality, coverage, defect metrics, and release readiness of **EasyService v1.0**.

Based on the verified local run:

1. **58 Maven tests** completed with zero failures and zero skips, including four live Selenium scenarios.
2. **JaCoCo Branch Coverage** reached **84.7%** for the core service package and passed the enforced 80% gate; whole-bundle coverage is 71.1% because it includes non-core classes.
3. The defect log records three fixed defects, but escaped-defect evidence and external CI build evidence are not yet complete.
4. GitHub Actions and Jenkins definitions exist; successful hosted runs still need to be attached as submission evidence.

FINAL RECOMMENDATION: READY FOR ACADEMIC PILOT/DEMONSTRATION; PRODUCTION RELEASE STILL REQUIRES PERSISTENT DATA, REGULATED PAYMENTS, SECURITY REVIEW, AND HOSTED CI/Jenkins evidence.

2. Testing Pyramid & Level Summary

- **Unit Testing (JUnit 5 + Mockito):** 35 unit tests validating business rules BR-01 to BR-20, equivalence partitions, boundary values, and decision tables.
 - **Integration Testing (Spring Boot Test + MockMvc):** 6 integration scenarios verifying REST controller execution and in-memory persistence workflows.
 - **System Testing (Selenium WebDriver):** Page Object Model journeys covering registration, verification, search, booking, and cancellation.
-

3. Foundations Reflection & Concept Synthesis

3.1 Verification vs. Validation

- **Verification:** "Are we building the product right?" Ensured through static code analysis, JaCoCo coverage enforcement, and unit tests adhering to BR-01..BR-20.
- **Validation:** "Are we building the right product?" Ensured through Selenium end-to-end user journeys validating real customer booking and provider listing workflows.

3.2 Error, Fault, Failure Spectrum

- **Error:** Developer misunderstanding of overbooking boundary conditions.
- **Fault:** Bug in `BookingTransactionService.java` using `>=` instead of `>`.
- **Failure:** System accepting booking quantity 101 when capacity was 100 during automated test run.

3.3 Test Doubles Selection Rationale

- **Fake:** `FakePaymentService` and `FakeIdentityVerificationService` provided full in-memory working logic with deterministic outcome toggling without real financial/identity exposure.
- **Mock/Spy:** `FakeNotificationService` tracked invocation counts to verify notification dispatches.
- **Controlled Randomness:** `FakeRandomNumberGenerator` eliminated non-deterministic randomness in the Wheel tool to enable reproducible test assertions.